Java 8 - AbstractReceiver et constructeurs

Objectif

Un exemple minimal reprenant le fonctionnement d'une classe abstraite avec constructeur sans paramètre, logger personnalisé et valeur transmise à `super(...)`.

Structure

```
message-receiver/
├── README.md
└── src/
    ├── InterfaceLogger.java
    ├── ConsoleLogger.java
    ├── Receiver.java
    ├── AbstractReceiver.java
    ├── MyReceiver.java
    ├── MyReceiver2.java
    └── Main.java
```

Classe de base

`Receiver` reçoit deux paramètres : un `InterfaceLogger` et une chaîne `type`.

```
public abstract class Receiver {
    private final InterfaceLogger logger;
    private final String type;

    protected Receiver(InterfaceLogger logger, String type) {
        this.logger = logger;
        this.type = type;
    }

    public void receive(String message) {
        logger.log(type + " : " + message);
    }
}
```

Classe abstraite intermédiaire

```
public abstract class AbstractReceiver extends Receiver {
    private static final InterfaceLogger LOGGER = new ConsoleLogger();

    public AbstractReceiver() {
        this(LOGGER);
```

```
    }

    public AbstractReceiver(InterfaceLogger specificLogger) {
        this(specificLogger, "DEFAULT");
    }

    public AbstractReceiver(InterfaceLogger specificLogger, String type) {
        super(specificLogger, type);
    }
}
```

- `AbstractReceiver()` conserve exactement le cas du constructeur sans paramètre.
- `this(LOGGER)` appelle le deuxième constructeur dans la même classe.
- `this(specificLogger, "DEFAULT")` définit la valeur par défaut.
- `super(specificLogger, type)` transmet le logger et la valeur à `Receiver`.

Classes filles

```
public class MyReceiver extends AbstractReceiver {
    public MyReceiver() {
        super();
    }
}
```

```
public class MyReceiver2 extends AbstractReceiver {
    public MyReceiver2() {
        this(new ConsoleLogger());
    }

    public MyReceiver2(InterfaceLogger logger) {
        super(logger, "SPECIAL");
    }
}
```

Chaque classe fille peut donc conserver les valeurs par défaut ou définir sa valeur directement dans `super`, sans `INSTANCE`.

Main

```
public class Main {
    public static void main(String[] args) {
        MyReceiver receiver1 = new MyReceiver();
        MyReceiver2 receiver2 = new MyReceiver2();

        receiver1.receive("Bonjour");
        receiver2.receive("Bonjour");
```

```
    }
}
```

Compilation Java 8

Sous Linux :

```
mkdir -p build
javac -source 8 -target 8 -d build src/*.java
java -cp build Main
```

Sous Windows CMD :

```
mkdir build
javac -source 8 -target 8 -d build src\*.java
java -cp build Main
```

Résultat attendu :

```
DEFAULT : Bonjour
SPECIAL : Bonjour
```

Aucune dépendance externe, ni Maven, ni Gradle ou Ant.